

ТЕХНИЧЕСКОЕ ЗАДАНИЕ · AI-АГЕНТ · V1.0

Модуль «Экспедиция» SaldaCargo ERP

Полная спецификация для AI-кодинг-агента: схема БД, API-эндпоинты, UI-компоненты, интеграция с финансовым контуром, документация.

Промт написан с учётом всех архитектурных конвенций проекта (CLAUDE.md).

ИП Нигамедьянов А.С. · г. Верхняя Салда · Сентябрь 2026 · Конфиденциально

ИНСТРУКЦИЯ

Как использовать этот документ

ПРИМЕНЕНИЕ

1. Скопируй текст промта (разделы 1–9) целиком
2. Вставь как задачу в AI-агента (Claude Code, Cursor, Gemini и т.д.)
3. Агент должен работать в репозитории `c:\salda`
4. Агент читает `CLAUDE.md` и wiki, затем создаст миграцию → API → UI → документацию
5. После завершения — проверь SQL-миграцию вручную перед применением к БД

ЕСЛИ АГЕНТ ЗАПРАШИВАЕТ УТОЧНЕНИЯ

- **Кошелёк для дохода экспедиции** → P/с ИП (UUID `00000000-0000-0000-0001-000000000001`)
- **Категория в навигации** → после «Ревью рейсов», перед «Финансы / P&L»
- **Валюта** → только рубли (₽)
- **Множественные заказы** → один перевозчик может иметь много заказов

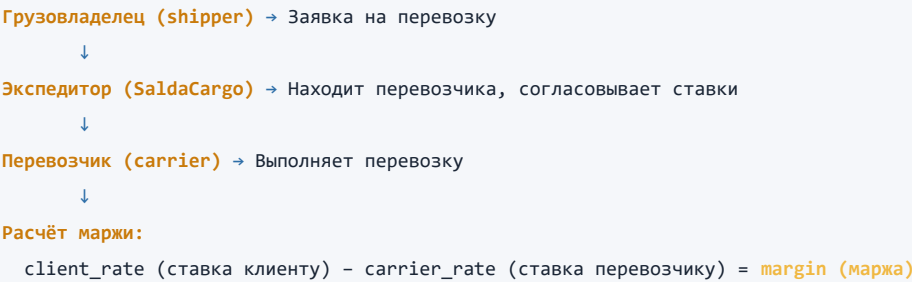
Обязательное чтение перед работой

- `CLAUDE.md` — главные правила проекта
- `kb/wiki/index.md` → `kb/wiki/Справочник_таблиц_БД.md` — текущая схема БД
- `kb/wiki/Транзакции.md` — как работают деньги в системе
- `kb/wiki/CrossApp_Features.md` — синхронизация приложений
- `kb/wiki/Миграции_БД.md` — регламент миграций
- `apps/web/app/(dashboard)/layout.tsx` — навигация WebApp

РАЗДЕЛ 01

Бизнес-логика экспедиции

Суть процесса



Статусы экспедиторского заказа (lifecycle)



СТАТУС	ЗНАЧЕНИЕ
new	Заявка создана, перевозчик не назначен
assigned	Перевозчик назначен, ожидает загрузки
in_transit	Груз в пути
delivered	Груз доставлен, ожидает финансового закрытия
completed	Финансово закрыт (все деньги прошли)
cancelled	Отменён (soft-delete, с причиной)

Финансовый контур (две оси, как везде в системе)

💰 Доход от клиента

- При создании заказа → income-транзакция
EXPEDITION_INCOME
- lifecycle=draft , settlement=pending
- При подтверждении оплаты →
settlement=completed

👤 Расход на перевозчика

- При назначении → expense-транзакция
EXPEDITION_CARRIER_PAY
- lifecycle=draft , settlement=pending
- При оплате перевозчику → settlement=completed

Маржа = income – expense по конкретному заказу.

РАЗДЕЛ 02

Схема БД — миграция

supabase/migrations/YYYYMMDD000001_expedition_module.sql

Таблица `expedition_carriers` — Перевозчики

```
CREATE TABLE expedition_carriers (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        TEXT NOT NULL,  
  contact_person TEXT,  
  phone       TEXT,  
  ati_id      TEXT,                -- ID на ATI.SU  
  ati_rating  DECIMAL(3,1),        -- рейтинг (0.0-5.0)  
  vehicle_types TEXT[],            -- ['газель', 'валдай', '10т', '20т']  
  regions     TEXT[],              -- ['Свердловская', 'Челябинская']  
  inn         TEXT,  
  notes       TEXT,  
  reliability  TEXT DEFAULT 'unknown' -- unknown/reliable/risky/blacklisted  
    CHECK (reliability IN ('unknown', 'reliable', 'risky', 'blacklisted')),  
  is_active   BOOLEAN NOT NULL DEFAULT true,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Таблица `expedition_orders` — Экспедиторские заказы

```
CREATE TABLE expedition_orders (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Грузовладелец (клиент)  
  shipper_id   UUID REFERENCES counterparties(id),  
  shipper_name TEXT,                -- для разовых клиентов без контрагента  
  shipper_phone TEXT,  
  
  -- Перевозчик  
  carrier_id   UUID REFERENCES expedition_carriers(id),  
  carrier_name_snap TEXT,          -- снимок имени на момент назначения  
  
  -- Маршрут  
  origin_city  TEXT NOT NULL,  
  destination_city TEXT NOT NULL,  
  route_description TEXT,  
  distance_km  INT,  
  
  -- Груз  
  cargo_description TEXT NOT NULL,  
  cargo_weight_kg DECIMAL(10,2),  
  cargo_volume_m3 DECIMAL(10,2),  
  vehicle_type  TEXT,              -- требуемый тип ТС  
  
  -- Даты  
  loading_date  DATE,
```

```

delivery_date    DATE,
actual_delivery  DATE,

-- Финансы (DECIMAL, как в CLAUDE.md)
client_rate      DECIMAL(12,2) NOT NULL,
carrier_rate     DECIMAL(12,2),
margin           DECIMAL(12,2) GENERATED ALWAYS AS
  (client_rate - COALESCE(carrier_rate, 0)) STORED,
margin_percent   DECIMAL(5,2) GENERATED ALWAYS AS (
  CASE WHEN client_rate > 0
    THEN ROUND((client_rate - COALESCE(carrier_rate,0))
      / client_rate * 100, 2)
    ELSE 0 END
  ) STORED,

-- Оплата
payment_method   TEXT DEFAULT 'bank_transfer'
  CHECK (payment_method IN ('bank_transfer','cash','card')),
client_paid      BOOLEAN NOT NULL DEFAULT false,
carrier_paid     BOOLEAN NOT NULL DEFAULT false,

-- Связь с транзакциями
income_tx_id     UUID REFERENCES transactions(id),
expense_tx_id    UUID REFERENCES transactions(id),

-- Статусы
status           TEXT NOT NULL DEFAULT 'new'
  CHECK (status IN ('new','assigned','in_transit',
    'delivered','completed','cancelled')),
cancelled_reason TEXT,

ati_order_id     TEXT,           -- номер заявки на ATI
notes            TEXT,
created_by       UUID REFERENCES users(id),
idempotency_key  UUID UNIQUE DEFAULT gen_random_uuid(),
created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Таблица `expedition_order_documents` — Документы

```

CREATE TABLE expedition_order_documents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id    UUID NOT NULL REFERENCES expedition_orders(id) ON DELETE CASCADE,
  doc_type    TEXT NOT NULL
    CHECK (doc_type IN ('тtn','заявка','договор','акт','счёт','прочее')),
  file_name   TEXT NOT NULL,
  storage_path TEXT NOT NULL,
  file_size   INT,
  mime_type   TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Таблица `expedition_order_history` — Лог статусов

```

CREATE TABLE expedition_order_history (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id    UUID NOT NULL REFERENCES expedition_orders(id) ON DELETE CASCADE,

```

```
old_status TEXT,  
new_status TEXT NOT NULL,  
changed_by UUID REFERENCES users(id),  
note TEXT,  
created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Категории транзакций

```
INSERT INTO transaction_categories (id, code, name, direction, is_system) VALUES  
(gen_random_uuid(), 'EXPEDITION_INCOME',  
'Доход экспедиции', 'income', true),  
(gen_random_uuid(), 'EXPEDITION_CARRIER_PAY',  
'Оплата перевозчику', 'expense', true);
```

Обязательные элементы миграции

- Комментарий с rollback в начале файла
- Триггеры `trg_..._updated_at` для carriers и orders
- `ALTER TABLE ... ENABLE ROW LEVEL SECURITY;` для всех 4 таблиц
- `CREATE POLICY "..._service_role_all"` с `USING (true) WITH CHECK (true)`
- `GRANT SELECT, INSERT, UPDATE ON ... TO service_role;`
- Индексы: `expedition_orders(status)`, `(shipper_id)`, `(carrier_id)`, `(loading_date)`

РАЗДЕЛ 03

API-эндпоинты (WebApp)

Все API в `apps/web/app/api/expedition/`. Паттерн: `createAdminClient()`, `(supabase.from('...') as any)`, Zod-валидация body.

GET /api/expedition/orders

Параметры: `?status=`, `?month=YYYY-MM`, `?carrier_id=`, `?limit=`

Возвращает список заказов с join на carrier и shipper (counterparty).

POST /api/expedition/orders

Создаёт новый заказ. Обязательно: `origin_city`, `destination_city`, `cargo_description`, `client_rate`.

Опционально создаёт income-транзакцию при `create_income_tx: true`. Idempotency через `idempotency_key`.

PATCH /api/expedition/orders/[id]

Обновление заказа. При назначении перевозчика (`carrier_id` впервые):

- `status` → `assigned`
- Записать `carrier_name_snap`
- INSERT в `expedition_order_history`

PATCH /api/expedition/orders/[id]/status

Смена статуса с валидацией переходов:

- `new` → `assigned` (нужен `carrier_id`) | `cancelled`
- `assigned` → `in_transit` | `cancelled`
- `in_transit` → `delivered`
- `delivered` → `completed`

При `cancelled` — требуется `cancelled_reason`. При каждой смене → INSERT в `expedition_order_history`.

POST /api/expedition/orders/[id]/pay-client

Отметить: клиент заплатил.

- `client_paid = true`
- Если `income_tx_id` есть → `settlement_status = 'completed'`
- Иначе → создать income-транзакцию `EXPEDITION_INCOME`, сразу `approved + completed`

POST /api/expedition/orders/[id]/pay-carrier

Отметить: перевозчику заплачено.

- `carrier_paid = true`
- Создать expense-транзакцию `EXPEDITION_CARRIER_PAY`
- `from_wallet_id` — передаётся в body

GET /api/expedition/carriers

Список перевозчиков. Параметры: `?search=`, `?reliability=`

POST /api/expedition/carriers

Создать перевозчика. PATCH `/carriers/[id]` для обновления.

GET /api/expedition/stats

Статистика за период (`?month=YYYY-MM`):

`total_orders`, `active_orders`, `total_revenue`, `total_cost`, `total_margin`, `avg_margin_percent`,
`top_carriers`, `top_routes`

РАЗДЕЛ 04

UI — WebApp Dashboard

Навигация

В `apps/web/app/(dashboard)/layout.tsx` → массив `navItems` — добавить:

```
{ href: '/expedition', label: 'Экспедиция', icon: 'swap_horiz' },
```

Вставить **после** `review` и **перед** `finance`.

Страница `/expedition` — 4 вкладки

```
apps/web/app/(dashboard)/expedition/page.tsx
```

Вкладка «Заказы» (orders) — по умолчанию

- Фильтры: статус (чипы), период (месяц ← →), поиск (город / клиент)
- Таблица: дата, маршрут, клиент, перевозчик, ставка клиенту, ставка перевозчику, маржа (₽ и %), статус
- Бейджи: `new` `assigned` `in_transit` `delivered` `completed` `cancelled`
- Клик по строке → слайдер с деталями
- Кнопка «+ Новый заказ» → модалка
- 3 сводных карточки: Активные заказы | Выручка/мес. | Маржа/мес.

Вкладка «Перевозчики» (carriers)

- Таблица: имя, телефон, АТИ ID, рейтинг, типы машин, регионы, надёжность (бейдж)
- Фильтр по надёжности, кнопка «+ Добавить перевозчика»
- Клик → карточка перевозчика с историей заказов

Вкладка «Аналитика» (stats)

- Период: месяц ← →
- KPI-карточки: Заказов, Выручка, Расходы, Маржа (₽), Маржа (%)
- График маржи по месяцам (bar chart)
- Таблицы: топ-5 маршрутов, топ-5 перевозчиков

Вкладка «P&L Экспедиция» (pnl)

МЕСЯЦ	ЗАКАЗОВ	ВЫРУЧКА	РАСХОДЫ	МАРЖА	%
-------	---------	---------	---------	-------	---

Данные из `expedition_orders` со статусом `completed` за 6 месяцев

Форма создания заказа (react-hook-form + Zod)

Обязательные поля

- `origin_city` — Город отправления
- `destination_city` — Город назначения
- `cargo_description` — Описание груза
- `client_rate` — Ставка клиенту

Оptionальные поля

- Клиент: выпадающий из counterparties ИЛИ ручной ввод
- Перевозчик: из `expedition_carriers`
- Даты: загрузка, доставка
- Вес, объём, тип ТС, оплата, ATI ID, примечания

Компоненты

`apps/web/components/expedition/`

`ExpeditionOrdersTab.tsx`

`ExpeditionCarriersTab.tsx`

`ExpeditionStatsTab.tsx`

`ExpeditionPnlTab.tsx`

`CreateOrderModal.tsx`

`OrderDetailSlider.tsx`

`CreateCarrierModal.tsx`

`CarrierDetailCard.tsx`

РАЗДЕЛ 05

Интеграция с финансовым контуром

В P&L (страница /finance)

Расходы `EXPEDITION_CARRIER_PAY` и доходы `EXPEDITION_INCOME` автоматически попадут в общий P&L через существующий API `/api/finance`. **Никаких изменений в `/api/finance` НЕ нужно.**

В дебиторке

Если клиент не заплатил (`client_paid=false` , `income_tx.settlement=pending`), заказ появится в дебиторке. Income-транзакция: `approved + pending` .

В группировке расходов/доходов

Добавить в `EXPENSE_GROUPS` :

```
'Экспедиция': [ 'Оплата перевозчику', 'EXPEDITION'],
```

Добавить в `INCOME_GROUPS` (аналогично):

```
'Экспедиция': [ 'Доход экспедиции', 'EXPEDITION'],
```

РАЗДЕЛ 06

Документация

ФАЙЛ	ДЕЙСТВИЕ
<code>kb/wiki/Справочник_таблиц_БД.md</code>	Добавить секцию «Модуль экспедиции» с 4 таблицами и 2 категориями
<code>kb/wiki/WebApp_Экспедиция.md</code>	Создать — суть, вкладки, API, бизнес-логика статусов
<code>kb/wiki/CrossApp_Features.md</code>	Добавить секцию «Экспедиция» — пока только WebApp

РАЗДЕЛ 07

Порядок выполнения

- 1 **Миграция БД** → `supabase/migrations/YYYYMMDD000001_expedition_module.sql`
- 2 **Обновить типы** → `pnpm db:types` (если возможно) или TODO
- 3 **API carriers** → CRUD `apps/web/app/api/expedition/carriers/`
- 4 **API orders** → CRUD + status transitions `.../expedition/orders/`
- 5 **API stats** → `apps/web/app/api/expedition/stats/`
- 6 **UI компоненты** → `apps/web/components/expedition/`
- 7 **Страница** → `apps/web/app/(dashboard)/expedition/page.tsx`
- 8 **Навигация** → обновить `layout.tsx` (добавить в navItems)
- 9 **Интеграция финансов** → добавить группу в EXPENSE_GROUPS / INCOME_GROUPS
- 10 **Документация** → обновить wiki

РАЗДЕЛ 08

Критичные правила (из CLAUDE.md)

⚠ НЕ НАРУШАТЬ

- **Деньги** — только строкой с двумя знаками: `"5000.00"`, в БД `DECIMAL(12,2)`
- **Supabase: всегда** `createAdminClient()`, `(supabase.from('...') as any)`
- **Балансы не хранятся** — вычисляются на лету
- **Soft-delete** через `status='cancelled'` + `cancelled_reason`, не DELETE
- **Idempotency** — `idempotency_key` на каждой форме, обработка 23505
- **UI на русском**, код на английском
- **Транзакции** — двухосный статус: `lifecycle_status` + `settlement_status`
- **Auth** — cookie `salda_user_id`, не Supabase Auth
- **Не дублировать** функции MiniApp ↔ WebApp

РАЗДЕЛ 09

Что НЕ делать в этой задаче

🚫 ЗАПРЕЩЕНО

- ❌ Не менять существующие таблицы (trips, trip_orders, transactions и т.д.)
- ❌ Не добавлять экспедицию в MiniApp (только WebApp на первом этапе)
- ❌ Не интегрировать с АТІ API (только поле `ati_order_id` для ручного ввода)
- ❌ Не создавать автоматическую рассылку уведомлений
- ❌ Не менять layout.tsx кроме добавления одного элемента в navItems
- ❌ Не рефакторить существующие компоненты
- ❌ Не удалять и не менять существующий код без явного запроса

SaldaCargo · ИП Нигамедьянов · г. Верхняя Салда

Промт для AI-агента · Модуль «Экспедиция» · v1.0 · Сентябрь 2026 · Конфиденциально